PROJETO 1 / C++ DESKTOP

CESEB

Relatório do projeto

Da necessidade do usuário à entrega
de uma calculadora Windows e Linux.

Requisitos · pesquisa · arquitetura · decisões
implementação · testes · correções · publicação

25 requisitos verificados
2 plataformas · Price + SAC
taxa nominal + efetiva

Código avaliado: 3f4dc69
Corte documental: b68e60a
Entrega: PR #1 · GitHub / paulo-vitor-cs/ceseb

RELATÓRIO DE TRABALHO / 01

Uma entrega verificável

Resumo executivo e identificação do trabalho

O CESEB é uma calculadora desktop local de financiamento, desenvolvida em C++20 com Qt Widgets. Permite informar valor financiado, taxa anual e prazo; selecionar a convenção nominal ou efetiva; e comparar os sistemas Price e SAC sob as mesmas condições. A interface está em português, com valores em reais e ajuda local.

Este relatório explica o produto e o trabalho de engenharia realizado entre 26 e 27 de outubro de 2026. Reconstrói a coleta de necessidades, o refinamento dos requisitos, as pesquisas, a arquitetura, as decisões, os planos, a implementação, a verificação e as correções. O ponto de partida foi um workspace sem código; o resultado é um aplicativo empacotado e um repositório público pessoal.

Dimensão	Resultado registrado
Produto	Cálculo local; sem conta, servidor, banco de dados ou persistência de simulações.
Requisitos	25 requisitos efetivos pass na revisão independente 2441.
Arquitetura	Quatro specs efetivas implemented; núcleo consumível sem Qt.
Testes	27/27 por backend e 21/21 core em cada plataforma; sem falhas ou skips nos ensaios finais.
Uso real	Pacotes usados em Windows 11 e Linux Ubuntu 24/WSL, em 100% e 200%.
Publicação	Código e Synapses publicados na branch codex/mortgage-desktop; PR #1 criada sob autorização humana.

O código realmente avaliado é **3f4dc69**. O corte **b68e60a** acrescenta documentação e recibos, sem alterar os 57 arquivos atuais de produto/testes. Estes dois novos documentos são um delta documental posterior: não representam um novo build ou uma nova execução dos binários.

Referências centrais: rodada final 2440; revisão 2441; reconciliação 2451.

RELATÓRIO DE TRABALHO / 02

A necessidade do usuário

Do pedido inicial às escolhas explícitas de escopo

> “Funcional para Windows e Linux em C++, com interface simples, porém moderna.”

O usuário solicitou desenvolver a proposta de projeto 1 da imagem fornecida, usando Mikoshi para construir specs, requisitos e os demais artefatos. A imagem serviu como contexto do desafio; seus textos não foram tratados como autorização para executar ações externas. A criação de um repositório público na conta pessoal foi expressamente solicitada.

A descoberta 1003 transformou o pedido em cenários: simular um financiamento; corrigir entradas inválidas; experimentar juros zero e limites; reproduzir builds e testes; e abrir um pacote sem preparar um ambiente de desenvolvimento. A proposta inicial foi submetida ao usuário antes da arquitetura concreta.

Questão de escopo	Escolha recebida e consequência
Convenção da taxa	Após explicação, o usuário pediu ambas: nominal anual com capitalização mensal e efetiva anual. O formulário precisa explicitar a convenção.
Amortização	Price e SAC, para comparar pagamentos ordinários e decrescentes, com ajustes de centavos explícitos.
Interface	O usuário preferiu o caminho mais simples. Qt Widgets/Fusion foi mantido; QML e animações não entraram no recorte.
Idioma e moeda	Design aprovado em pt-BR/BRL, com parsing, rótulos e mensagens consistentes.
Plataformas	Windows e Linux; posteriormente, WSL foi aceito explicitamente para o cenário Linux do requisito 1226.
Repositório e integração	Conta pessoal pública; push autorizado e executado. Neste pedido posterior, PR e merge foram autorizados.

Ficaram fora do recorte: tabela mensal exposta na interface, CSV, seguros, tarifas, impostos, indexadores, serviços bancários, login e sincronização. Os livros mensais existem internamente para o cálculo e sua verificação; isso não implica uma tela de tabela de amortização.

Fontes: descoberta 1003 e respostas e emendas 1005.

RELATÓRIO DE TRABALHO / 03

O processo seguido

Mikoshi Design com gates, pesquisa e Synapses

O trabalho foi conduzido em sete fases. Um gate é uma autorização sobre um conjunto concreto; não é aprovação antecipada de qualquer conteúdo futuro. O usuário aprovou avanços, revisões e planos em momentos distintos. Autoria, execução, julgamento e publicação foram tratados como fatos diferentes.

Fase	O que ocorreu no projeto
1 · Descoberta	Necessidade, atores, cenários, limites, alternativas e proposta de escopo 1003.
2 · Exploração	Leitura do workspace e das dependências; inicialmente não havia código ou toolchain completa no ambiente observado.
3 · Esclarecimentos	Definição de nominal/efetiva, Price/SAC, interface simples, idioma e domínio de entrada.
4 · Arquitetura	Quatro contratos; refinamento dos requisitos; pesquisas financeira, técnica e de distribuição; revisão numérica e do Qt.
5 · Planejamento e execução	Planos distintos de testes e produto, fixtures independentes, preparação de dependências, builds, pacotes e procedimentos.
6 · Synapses	Revisão independente, resultados por requisito, aplicação dos verdicts pelo analyst e preservação das versões.
7 · Publicação	Syncer, commits e push com leitura direta do remoto; posteriormente, PR e merge sob a nova autorização.

O fluxo também precisou de correções

O usuário apontou que os primeiros requisitos eram esboços, com formatação inadequada e prescrições de ferramentas. O design foi preservado e os requisitos foram refeitos. A pesquisa estruturada também foi exigida quando a consulta inicial de fontes ainda não demonstrava o fluxo completo.

Houve ainda omissão na atualização de maturidade após aprovação e uma interpretação indevida que limitou a publicação ao local. As correções registraram a aprovação aplicável e carregaram a ordem de push até o syncer. O relatório mantém esses fatos como lições do trabalho, sem reescrever o histórico como um fluxo sem desvios.

RELATÓRIO DE TRABALHO / 04

Da necessidade ao requisito

Estrutura verificável, nível, rationale e critérios de aceite

A necessidade humana define o resultado desejado. A spec delimita uma funcionalidade e sua fronteira. O requisito descreve uma obrigação observável. O plano escolhe os meios, as operações por arquivo e o procedimento para implementá-la ou testá-la. Ferramentas são contratos operacionais referenciados pelos planos; não são propriedades funcionais inventadas do produto.

Campo do requisito	Função
Requirement	Comportamento exigido, com unidade e condição de aplicação.
Origin and Boundary	Origem humana, spec de contexto e nível: unidade, integração ou sistema.
Rationale	Por que a obrigação é necessária e qual falha evita.
Acceptance Criteria	Condições determinísticas que definem aprovação e reprovação.
Procedure / Oracle	Entradas, passos, resultado esperado e tolerância quando aplicável.
Trace	Relação recuperável com origem, revisão, planos, arquivos e evidência.

Exemplo real: requisito 1102, conversão nominal

Obrigação: com taxa nominal anual válida, produzir a fração mensal p/1200. **Nível:** unidade. **Rationale:** evitar confusão entre percentual anual, fração e período.

Critérios: para 0%, 4,5%, 12% e 100% anuais, obter 0; 0,00375; 0,01; e 1/12. A diferença não nula admitida na projeção é no máximo 1e-12 absoluto; zero deve ser exato. O teste compara números de referência, não texto da UI.

Rota: necessidade 1005 → spec de taxa → requisito 1102 → T1102/I-1102 nos planos 2001/2002 → finance.cpp e teste de domínio → resultado por requisito em 2441. A tolerância de projeção não decide centavos de pagamentos.

Os antigos 1222–1224, que prescreviam Google Test, QtTest e CI, ficaram obsolete. Seus meios foram documentados nos cartões 2302–2304 e usados nos planos. A cobertura efetiva permaneceu em 25 requisitos de produto, com 11 de unidade, seis de integração e oito de sistema. Fonte: refinamento 1018.

RELATÓRIO DE TRABALHO / 05

O catálogo efetivo

25 obrigações de produto; resultado final pass no código 3f4dc69

REQ	Nível	Obrigação resumida	Final
1101	Unidade	Identificar explicitamente a convenção da taxa.	pass
1102	Unidade	Converter nominal anual em fração mensal.	pass
1103	Unidade	Converter efetiva anual em mensal equivalente.	pass
1104	Unidade	Produzir equivalentes anuais coerentes.	pass
1105	Unidade	Rejeitar taxas não finitas ou fora do domínio.	pass
1106	Integração	Preservar origem exata da taxa para o cálculo.	pass
1201	Unidade	Validar principal, prazo e ordem dos erros.	pass
1202	Unidade	Calcular Price, quitação e ajuste final.	pass
1203	Unidade	Calcular SAC e distribuir centavos indivisíveis.	pass
1204	Unidade	Conservar principal e reconciliar os totais.	pass
1205	Unidade	Tratar juros zero e vencimento único.	pass
1206	Unidade	Recusar casos sem a precisão mínima de centavos.	pass
1207	Integração	Comparar os dois livros sob a mesma entrada.	pass
1211	Integração	Interpretar entradas brasileiras sem ambiguidade.	pass
1212	Integração	Apresentar taxas com seis casas e half-up.	pass
1213	Sistema	Mostrar resumos, contagens e pagamentos finais.	pass
1214	Integração	Invalidar resultados e recuperar o estado da UI.	pass
1215	Sistema	Operar formulário, ajuda e fechamento por teclado.	pass
1216	Sistema	Alcançar e ler controles nas dimensões/escalas.	pass
1217	Sistema	Disponibilizar ajuda local e versão identificada.	pass
1221	Integração	Compilar e consumir o núcleo sem Qt.	pass
1225	Sistema	Entregar pacotes executáveis nas plataformas.	pass
1226 r2	Sistema	Executar cenários essenciais Windows e Linux/WSL.	pass
1227	Sistema	Entregar documentação e relatório rastreáveis.	pass
1228	Sistema	Preservar licenças e avisos dos componentes reais.	pass

As quatro specs efetivas agrupam 6 + 7 + 7 + 5 requisitos: taxas 1008 r2; amortização 1012 r2; UI 1013 r2; entrega 1014 r3. A revisão final usa 1226 r2. Referências antigas nos corpos e planos históricos conservam seu significado original.

RELATÓRIO DE TRABALHO / 06

Pesquisa que mudou o design

Três frentes, 19 notas e decisões discriminantes

A pesquisa não se limitou a procurar uma biblioteca que compilasse. Foram organizadas três frentes com fontes, unidades efetivamente lidas, contraevidência, síntese e limites. As 19 notas produzidas continuam draft: publicação no Git não lhes atribui automaticamente maturidade de conhecimento consolidado.

Frente	Perguntas e efeitos no projeto
Financeira · 3001	Convenções anual/mensal, Price/SAC e centavos; identificou que proximidade decimal não prova o lado correto de um empate. Cinco notas.
Técnica · 3002	Parsing, formatação, precisão Boost, QtTest, runner e isolamento do núcleo; seleção vazia de testes não vale como sucesso. Seis notas.
Distribuição · 3003	Acesso e manutenção Qt, kits, plataformas, dependências runtime, CRT, licenças e evidência de pacote. Oito notas.

Duas revisões materiais aprovadas

Precisão monetária: a proposta inicial de decimal de 50 dígitos era insuficiente para prometer decisão exata em todos os empates do contrato. A análise 1023 comparou aumento de precisão, epsilon, taxa mensal quantizada e aritmética exata. Foi escolhida a representação racional e a comparação/refinamento de raízes, sem alterar a regra half-up.

Qt: a baseline passou de 6.8.3 para o kit oficial fixado em 6.12.0. A análise 1024 comparou manter o pin, aplicar patches próprios, compilar outras versões e usar uma rota comercial. A escolha preservou Widgets e os compiladores; não introduziu funcionalidades novas nem assinatura comercial.

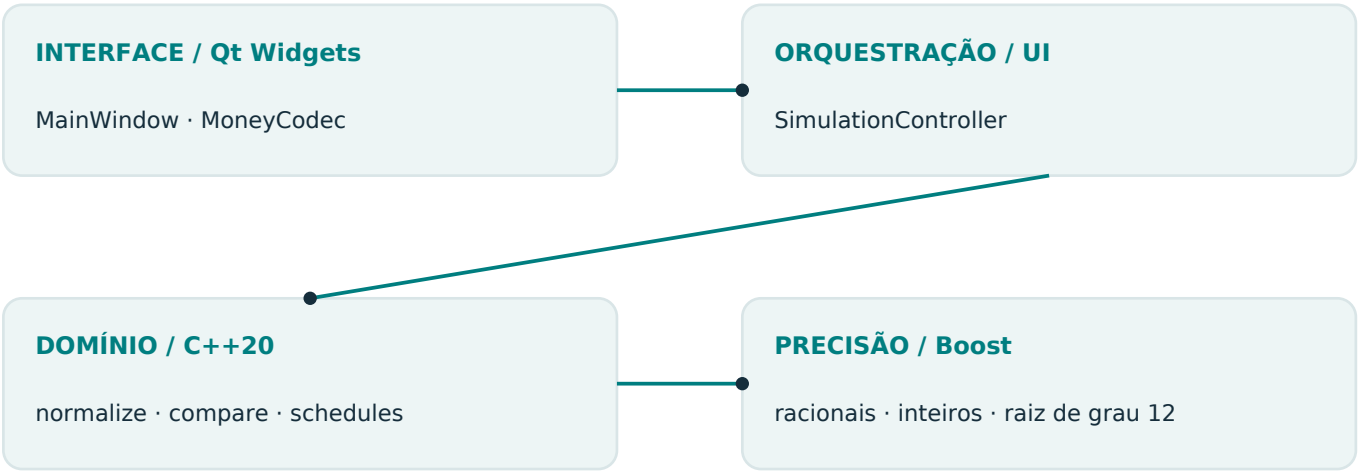
A consulta de fontes sustenta decisões, mas não demonstra runtime. Avisos conhecidos do Qt foram conservados na análise de superfície; não foi declarada segurança global. A disponibilidade de um kit também não provou, sozinha, ABI, licenciamento do pacote ou compatibilidade do aplicativo.

Fontes de análise: 1023 · método numérico e 1024 · baseline Qt.

RELATÓRIO DE TRABALHO / 07

Arquitetura em quatro partes

Separação entre apresentação, estado, domínio e precisão



Resultados e erros tipados retornam ao controlador; o núcleo não depende de Qt.

O formulário não calcula diretamente a partir de strings formatadas. MoneyCodec interpreta texto pt-BR; SimulationController coordena cálculo e invalidação; o núcleo recebe LoanInput tipado e produz Comparison ou Error. MainWindow apresenta resultados e mensagens, preservando a possibilidade de corrigir a entrada.

Componente / arquivo	Responsabilidade e fronteira
include/ceseb/finance.hpp	API pública: tipos de taxa, entrada, erro, cronograma e comparação. Não inclui Qt.
src/core/finance.cpp	Normalização, livros Price/SAC, invariantes, totais e rejeições do domínio.
src/core/exact_math.*	Intermediários inteiros/racionais, comparação de fronteiras, raízes e arredondamento.
src/ui/money_codec.*	Validação textual e formatação pt-BR; sem devolver rótulos ao cálculo.
src/ui/simulation_controller.*	Receber ações, invocar o núcleo, invalidar resultados e recuperar estado.
src/ui/main_window.cpp	Formulário, cartões, teclado, rolagem, ajuda e aparência da janela.

A fronteira independente do núcleo permite compilar e consumir o componente sem Qt. A seleção dos módulos Qt pertence ao build da interface; essa organização foi verificada separadamente dos testes de aparência. O diagrama representa dependências lógicas, não threads ou processos de servidor.

RELATÓRIO DE TRABALHO / 08

O modelo financeiro

Mesma entrada e convenções explícitas para Price e SAC

Entradas do primeiro recorte: principal entre R$ 0,01 e R$ 100.000.000,00; taxa anual entre 0% e 100%, inclusiva; prazo de 1 a 600 meses. O valor monetário entra no núcleo como inteiro de centavos; a convenção da taxa é um tipo explícito. O primeiro vencimento segue a periodicidade mensal do modelo, sem carência ou encargos adicionais.

Conversão de taxas

Se p é o percentual anual, a taxa mensal nominal é **i = p/1200**. Na convenção efetiva, **i = (1 + p/100)^(1/12) − 1**. Para a mesma taxa mensal, os equivalentes anuais são 1200i% nominal e 100[(1+i)^12 − 1]% efetivo. A diferença entre as convenções precisa permanecer visível ao usuário.

Price e SAC

Price usa a prestação teórica **A = C·i·(1+i)^n / [(1+i)^n − 1]** quando i é positivo; com i = 0, usa C/n. O livro em centavos aplica a parcela ordinária arredondada, calcula juros sobre o saldo e ajusta o último pagamento para quitar exatamente. O número efetivo de cobranças pode ser menor que o prazo informado.

No SAC, a amortização inteira é obtida pelo quociente do principal pelo prazo, distribuindo o resto de um centavo nas primeiras linhas. Cada prestação é amortização mais o juro arredondado do saldo anterior. O saldo fecha em zero e as somas precisam reconciliar exatamente.

R$ 1.000,00 · 12% nominal · 3 meses	Price	SAC
Primeira / ordinária	R$ 340,02	R$ 343,34
Última prestação	R$ 340,03	R$ 336,66
Total pago	R$ 1.020,07	R$ 1.020,00
Juros totais	R$ 20,07	R$ 20,00

Com 12% efetivos anuais, a mensal exibida é 0,948879%; os totais do mesmo exemplo são R$ 1.019,04 no Price e R$ 1.018,98 no SAC. Isso ilustra o produto implementado, não uma oferta de crédito ou uma convenção universal de instituição financeira.

RELATÓRIO DE TRABALHO / 09

Centavos não dependem de epsilon

Origem exata, comparação de fronteiras e apresentação separada

A aproximação decimal continua útil para projeções, mas não decide o centavo monetário. A taxa anual original é conservada como racional exato; dinheiro e somas são inteiros de centavos. Boost fornece inteiros e racionais de precisão arbitrária nos intermediários. O rounding exigido é **half-up**: um empate positivo sobe para o centavo seguinte.

Como o método decide

No ramo racional, uma fração não negativa u/v é arredondada por divisão inteira de (2u+v)/(2v), tomando o piso. Na taxa efetiva, o crescimento mensal x satisfaz x^12 = 1+p/100. Quando a raiz é racional, usa-se o ramo racional; nos demais casos, comparações de potência e multiplicação cruzada decidem limiares sem epsilon.

Para a prestação Price com raiz irracional, o método refina um intervalo racional até os dois extremos arredondarem ao mesmo centavo. A análise 1023 justifica a terminação para o contrato; a implementação não incorpora um motor algébrico genérico. Terminação matemática e custo observado são evidências diferentes.

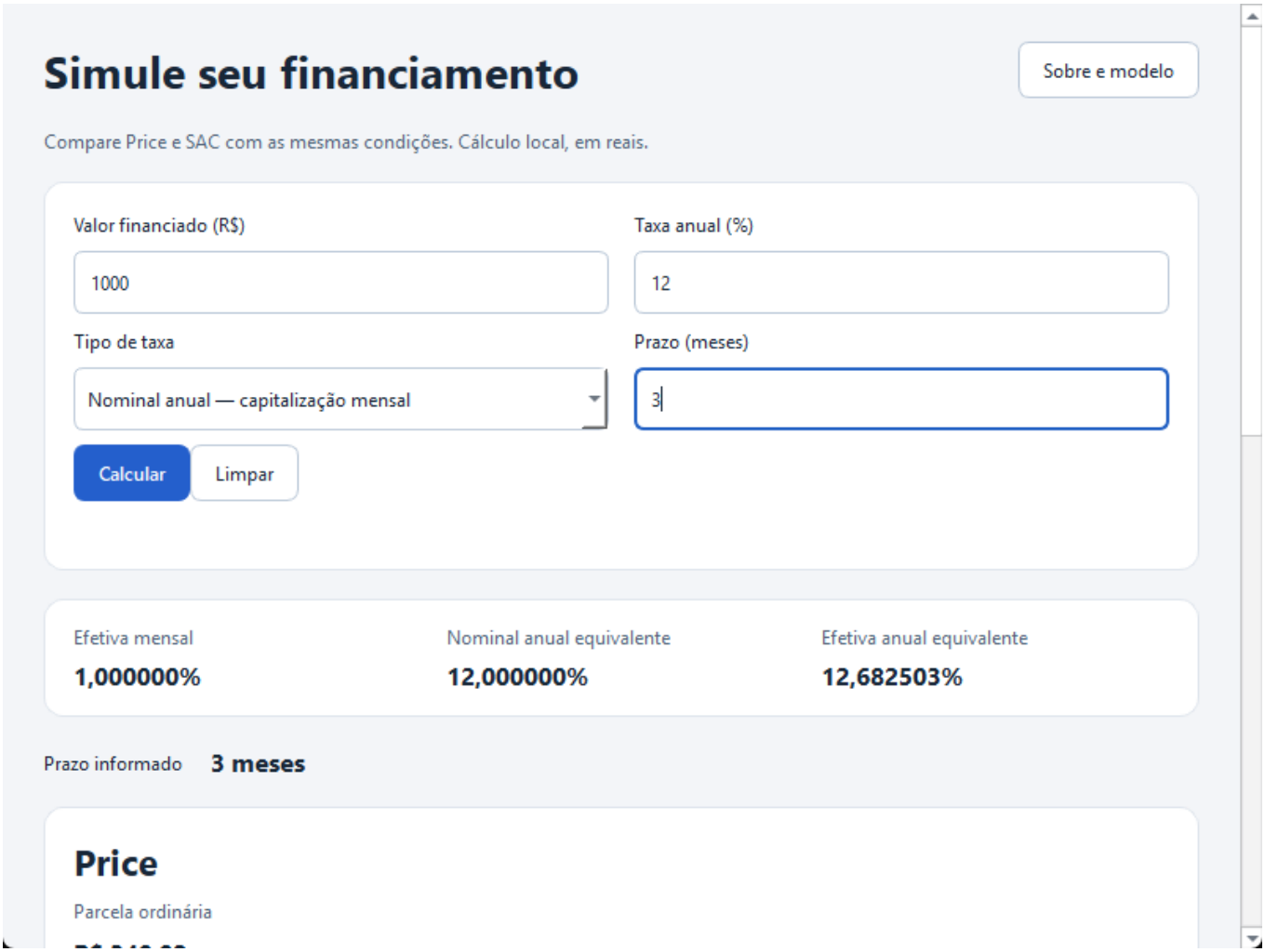
Caso decisivo	Comportamento verificado
R$ 0,06 · nominal anual 100% · 1 mês	Juro de R$ 0,01 e pagamento de R$ 0,07 em ambos os sistemas.
R$ 0,03 · zero · 2 meses	Price em centavos [2, 1], com quitação sem saldo residual.
R$ 1.000,00 · zero · 600 meses	Price: 599 pagamentos, ordinária R$ 1,67 e última R$ 1,34. SAC: 600 pagamentos, com centavos distribuídos.
Apresentação das taxas	Seis casas em milionésimos de ponto percentual, half-up explícito; o texto não retorna ao cálculo.

Casos que não atendem à amortização mínima de centavos definida pelo modelo retornam UnsupportedCentPrecision, sem comparação ou livro parcial. A interface apresenta erro recuperável. A tolerância dos testes de projeção de taxa não foi reutilizada como tolerância monetária.

RELATÓRIO DE TRABALHO / 10

Uma interface simples e utilizável

Formulário, cartões, estado explícito e ajuda offline



Captura original do pacote Windows corrigido. O sistema permaneceu com aparência escura; o app usa sua paleta clara. O restante dos resultados fica acessível por rolagem.

O formulário reúne valor, taxa, tipo e prazo. Calcular apresenta equivalentes de taxa e cartões Price/SAC; Limpar devolve o estado inicial. Qualquer edição invalida resultados anteriores. Entradas erradas preservam os dados necessários à correção e indicam o foco adequado.

A navegação inclui Tab e sequência inversa, Enter, Space, ajuda e Escape. O layout de 800×600 pixels lógicos permite alcançar os controles e resultados nas duas escalas verificadas. A ajuda explica convenções, arredondamentos, exclusões, versão e acesso às licenças locais. Não há exigência de rede durante a simulação.

RELATÓRIO DE TRABALHO / 11

Registro das decisões

O que foi escolhido, por quê e com que efeito

Registro	Estado	Decisão	Rationale / efeito
1007	decided	Ambas as convenções de taxa; tipo e período explícitos.	Evitar comparar percentuais anuais com significados diferentes.
1011	decided	Price/SAC, pt-BR/BRL, UI Widgets e limites de domínio.	Concentrar o primeiro produto em comparação local e operação simples.
1025	decided	Origem racional e fronteiras monetárias exatas.	Cumprir half-up nos empates sem epsilon ou quantização oculta.
1026	decided	Qt 6.12.0 oficial, mesmo Widgets/Fusion.	Usar kits fixados sem manter patches próprios ou ampliar a UI.
1027	decided	Integração das revisões numérica e Qt.	Preservar predecessores e relações, sem aprovação retroativa.
1032	decided	Planos de teste/produto distintos e execução em dois momentos.	Ter oráculos independentes e separar range de assets do binário.
1037	decided	WSL aceito para o cenário Linux de 1226.	Usar o ambiente disponível com autorização humana explícita.

As escolhas iniciais e a correção de requisitos também têm registros 1004 e 1018, ainda com metadata open. Esse estado foi preservado; este relatório não fecha decisões por inferência. As ações e aprovações posteriores têm suas próprias fontes de autoridade.

Na correção final, foram escolhidos dois meios locais: paleta completa no processo do app e a opção de deploy que omite DXC opcional. Não se alterou o tema global do Windows, a política de centavos ou o conjunto de critérios. Essas medidas estão nos planos 2005/2006 e na liberação concreta 1042.

As alternativas descartadas não viraram requisitos: QML, epsilon, aumento arbitrário de precisão como garantia, taxa mensal quantizada e patchset Qt próprio. Cada alternativa tinha um custo ou alterava um contrato já escolhido.

RELATÓRIO DE TRABALHO / 12

Planejar antes de alterar

Procedimento concreto, autoria e cobertura por arquivo

Os planos iniciais 2001 e 2002 cobrem as 25 obrigações, com uma tarefa por requisito e operações de arquivo explícitas. O documento 1030 acorda interfaces físicas e fixtures. O plano de testes veio antes do produto; em um base sem implementação, ausência de alvo foi registrada como indisponibilidade, não como um RED fictício.

Ciclo	Plano de testes	Plano de produto	Resultado preservado
Inicial	2001: 22 arquivos de assets; sete tarefas sem alteração de fonte.	2002: 31 arquivos novos; uma tarefa sem alteração adicional.	Corpos aprovados e retornos de execução separados.
Distribuição	2004: regressões e procedimentos pendentes.	2003: launcher DEB e documentação/links.	F1/F2 corrigidos; resultados anteriores mantidos.
F3/F4	2005: quatro operações de teste.	2006: dois arquivos de produto.	Regressões antes da correção; planos finais implemented.

O ciclo final tem identidades separadas

BASE público: b5bb61c. **TEST_HEAD:** 98af104. O range de assets b5..98 cobre quatro operações de teste. **PRODUCT_HEAD:** 3f4dc69; o range 98..3f4 cobre apenas main_window.cpp e WindowsDeploy.cmake. A execução final dos testes refere-se ao produto 3f4; não transforma o range de assets em um range de produto.

A correção final seguiu o arranjo explícito de autoria Astra e execução Sol 6.1. Autores/executores de testes e produto usaram heads e memórias distintos; o revisor independente foi outro participante. O analyst aplicou os resultados recebidos sem refazer o julgamento, e o syncer cuidou dos efeitos Git autorizados.

Cobertura de commit é necessária para declarar o plano implemented, mas não prova satisfação do requisito. Os planos antigos 2001–2004 mantêm seus status históricos partial implemented; os planos 2005/2006 estão implemented. O parecer final de produto é o conjunto de 25 pass no código 3f4, não uma reescrita desses planos antigos.

RELATÓRIO DE TRABALHO / 13

Implementação e reprodução

Núcleo primeiro, interface depois e dependências fixadas

Tecnologia	Versão / uso no projeto
C++20	Domínio, API tipada, inteiros de centavos e aplicação desktop.
Boost 1.86.0	Multiprecision header-only: cpp_int, cpp_rational e projeção decimal.
Qt 6.12.0	Core/Gui/Widgets dinâmicos; Fusion. Qt Test somente no desenvolvimento.
Google Test 1.17.0	Casos determinísticos do núcleo e fixtures independentes.
CMake ≥ 3.28 · Ninja · CTest	Configuração, build, inventários, execução e empacotamento.
Compiladores	MSVC 2022/v143 no Windows; GCC 13 no Ubuntu 24.04.
Entrega	ZIP Windows e DEB amd64 Ubuntu, com runtime privado e materiais de licença.

cmake/dependencies.lock.json fixa versões, URLs, commits e SHA256 dos arquivos de dependências. Aquisição, extração e preparação de notices foram separadas do código do produto. A rota core-only desabilita GUI e descoberta de Qt; a rota com UI compõe os prefixos reais do kit e executa seus testes.

As exportações nativas usam a revisão Git real como CESEB_SOURCE_REVISION e deixam build/output fora da fonte exportada. No Windows, os caminhos do SDK pertencem ao processo de desenvolvimento; o app empacotado foi aberto sem essas variáveis. No Linux, o runtime separado contém Qt privado, sem toolchain do compilador.

O DEB mantém runtime em /opt/ceseb e registra o launcher em /usr/share/applications. O ZIP mantém binários, plugins, CRT qualificado e notices. O conjunto documental histórico exigido pelo empacotamento tem 21 registros; não é uma atribuição automática desses testes antigos a qualquer pacote novo.

Reprodução e comandos completos: guia de build. Este guia contém contexto histórico; para o resultado corrente, consultar 2412, 2421 e 2441. O workflow GitHub Actions existe com disparo manual, mas nenhum run remoto foi observado: fonte de CI não equivale a CI executada.

RELATÓRIO DE TRABALHO / 14

Estratégia de verificação

Oráculos independentes e métodos adequados à obrigação

Camada / método	O que foi verificado
Domínio · Google Test	20 casos: convenções, limites, juros zero, prazo único, Price/SAC, empates, invariantes e rejeições.
Integração/UI · QtTest	Cinco slots finais: parsing/apresentação/estado e regressão de glifos sob paletas herdadas.
Inventários · CTest	Duas verificações adicionais na suíte GUI; seleção vazia não é sucesso.
Core isolado	20 casos de domínio mais inventário: 21/21 sem depender de Qt.
Inspeção de pacote	Hash, manifesto, notices/fontes, imports normais e delay-imports, dependências e launcher.
Demonstração	Uso real por teclado, nominal/efetiva, erros, correção, limpeza, ajuda, escala e fechamento.

A suíte final com interface contabiliza **27 = 20 casos de domínio + 5 slots GUI + 2 inventários**. O core contabiliza **21 = 20 casos + 1 inventário**. Esses números não se somam como 48 casos financeiros distintos. A grade determinística de 480 combinações amplia as entradas exercitadas; não é prova de exaustividade ou medição de cobertura de linhas.

Fixtures e valores de referência foram definidos antes do produto. Assert de centavo não passa por double; erro de ambiente não vira falha de comportamento; ausência de execução não vira pass. A revisão final repetiu as suítes em builds próprios e distinguiu as demonstrações recebidas de inputs que o próprio revisor não realizou.

O que as regressões novas provaram

O teste de pixels compara campo preenchido e vazio em regiões de texto. No base, a paleta clara tinha diferenças de glifos e a escura tinha zero nos quatro controles; no produto, ambos os pares passam. Isso prova presença de glifos, não um limiar de contraste inventado. A leitura das capturas reais forneceu a evidência visual complementar.

O checker do ZIP encontrou DXC/DXIL no pacote antigo e deixou de encontrá-los no novo. O inventário obrigatório permaneceu presente. A inspeção PE e os módulos em runtime confirmaram que a exclusão não foi mascarada por um fallback do SDK.

RELATÓRIO DE TRABALHO / 15

Validação nas duas plataformas

Builds próprios, app distribuído e limites do ambiente real

Condição final	Windows 11	Ubuntu 24.04 / WSL
Toolchain	Community 2022 / MSVC 19.44	GCC 13.3; userspace Ubuntu 24.04.5
Suíte offscreen	27/27	27/27
Suíte com backend nativo	27/27 · windows	27/27 · XCB
Núcleo separado	21/21	21/21
App empacotado	ZIP extraído; módulos app-local	DEB instalado no runtime separado
Uso em 100% e 200%	47/47 alvos por escala	47/47 alvos por escala
Falhas/skips finais	0 / 0	0 / 0

Linux usou contêineres privados: um para compilação Ubuntu 24/GCC 13 e outro para runtime, com X11/Xvfb/Openbox. O kernel é compartilhado com WSL. O runtime não tinha toolchain e foi observado offline, somente com interface lo e sem rota IPv4. Isso é desktop Linux virtualizado realmente usado, não uma máquina física Ubuntu independente.

Windows usou o host Windows 11 real. O SDK estava instalado em outra árvore, mas o processo do pacote executou sem caminhos de desenvolvimento. O tema escuro global permaneceu intacto. Entradas nativas foram guardadas por sessão desbloqueada, PID, EXE e janela ativa; outras janelas não receberam input.

Na escala 200%, o cliente de 1600×1200 excedia o monitor físico de 1920×1080 em altura. Apenas a janela própria foi movida e rolada para trazer as regiões à área visível. A aprovação visual usa CopyFromScreen de regiões reais, não pixels supostos de uma janela fora da tela. As quatro combinações sistema/escala foram inspecionadas.

A falha de inicialização do provedor Computer Use ocorreu antes de controlar o app. O usuário autorizou outro método; a validação usou APIs nativas e capturas reais. Essa falha do provedor não foi interpretada como falha funcional do CESEB.

RELATÓRIO DE TRABALHO / 16

Problemas encontrados e corrigidos

Um histórico útil inclui RED, diagnóstico e efeito do reparo

Finding	Falha observada	Correção e evidência
F1 · distribuição	Launcher apenas na árvore privada; descoberta desktop inadequada.	Registro em /usr/share/applications, com teste do launcher no DEB.
F2 · documentação	Links de licença quebrados e registros de evidência ausentes do pacote.	Licenças recuperáveis, links corrigidos e input documental explícito de 21 registros.
F3 · notices	ZIP incluía dxcompiler.dll/dxil.dll sem materiais específicos recuperáveis.	windeployqt com --no-system-dxc-compiler; ZIP real sem as DLLs, com fechamento PE/runtime conferido.
F4 · aparência	Texto branco sobre branco nos quatro campos sob Windows escuro; outros textos com contraste inadequado.	Paleta completa antes dos filhos, pares de cores/seleção e superfícies consistentes; uso real e regressão nas escalas.

No ciclo F3/F4, três requisitos estavam reprovados: 1216 por aparência e 1225/1228 pela mesma causa de pacote. Eram duas causas concretas, não três bugs independentes. Após reproduzir os defeitos, o produto alterou apenas main_window.cpp e WindowsDeploy.cmake. Os quatro arquivos de regressão permaneceram iguais entre BASE e FINAL.

Tentativas de preparo não foram apagadas

A primeira escrita do teste usou incorretamente o retorno void de setStyle; foi corrigido somente o preparo antes de contar o RED real. Houve também falhas de invocação/ferramentas de pacote, sincronização de observadores de GUI e guard de foco que interrompeu antes de enviar input a outra janela. Os registros diferenciam essas ocorrências das falhas do produto.

O princípio preservado foi reconciliar o efeito antes de tentar novamente, sem relaxar o oráculo, alterar preferência global ou declarar sucesso pelo simples retorno de uma ferramenta. Relatórios históricos não foram convertidos retroativamente em resultados verdes.

RELATÓRIO DE TRABALHO / 17

Distribuição e licenças

Pacotes reais, composição recuperável e identidade dos binários

Os binários finais registrados foram construídos no código 3f4dc69. A adição deste PDF e do HTML ao repositório não reconstrói esses pacotes. A versão é CESEB 0.1.0; não há release GitHub publicada neste corte. ZIP e DEB foram gerados e ensaiados localmente.

Pacote	Composição verificada
Windows ZIP	84.400.666 bytes; 188 arquivos, 187 hashes declarados; 22 imagens PE AMD64, sem imports/delay-imports obrigatórios irresolvidos. Sem DXC/DXIL.
Ubuntu DEB	97.400.034 bytes; 184 arquivos, 182 hashes declarados; Qt/ICU privados, dependências de sistema e launcher recuperáveis.
Materiais históricos	21 registros entregues por pacote, comparados à origem e atribuídos aos seus snapshots antigos.

SHA256 dos pacotes testados

```
ZIP: f5071e93d3c99afd4b5fae395322897c8d
074f928f66952366eb0b05ad762d20
```

```
DEB: 8dd414362fc06456a7aadaf44acb5a8026
4514497433d0ba24f240d129628336
```

MIT se aplica aos arquivos originais do projeto. Qt, ICU, Boost, Google Test, CRT e terceiros conservam seus próprios termos. A entrega inclui fonte Qt correspondente, configuração, SBOM e notices; não basta renomear tudo como MIT. A revisão confrontou a composição real com os materiais disponíveis.

No Windows, oito CRT Release foram qualificados para a instalação Community observada. A árvore completa licenses possui 132 arquivos no ZIP e 130 no DEB; o denominador 127 refere-se a um subconjunto de originais com hashes, não à contagem total da árvore. A ligação dinâmica e a configuração foram inspecionadas; não se afirma ensaio ABI ou certificação jurídica.

RELATÓRIO DE TRABALHO / 18

Resultados e entrega do trabalho

Verificação, reconciliação e publicação são etapas diferentes

O parecer independente 2441 julgou os 25 requisitos pass no mesmo código 3f4. O analyst 2451 executou 31 operações nativas: 25 requisitos, quatro specs e dois planos. Houve sete mudanças literais e 24 reafirmações; os planos finais 2005/2006 ficaram implemented. A conclusão de um plano não foi usada como substituto de um resultado de requisito.

Objeto	Preservação / resultado
49 corpos aprovados	Os 47 valores anteriores mais os dois planos novos, sem reescrever corpos aceitos.
57 paths atuais	Produto/testes coincidem com Git 3f4; seis paths mudaram no ciclo corretivo e dois deles são novos testes.
38 arquivos históricos	Relatórios publicados anteriores preservados byte a byte.
19 notas de pesquisa	Continuam draft; sem promoção de conhecimento por inferência.
Publicação anterior	183fabed publicou o recorte de resultados; b68e60a publicou o relatório do push e o fechamento documental.
Integração atual	PR #1 criada; PR e merge expressamente autorizados pelo usuário em 27/10/2026.

A branch codex/mortgage-desktop teve HEAD local e ref remota conferidos diretamente em b68e60a ao final da publicação Synapses. A PR de entrega reúne esse trabalho e, como delta documental posterior, este relatório e a apresentação. O estado de integração e o commit da PR são consultáveis no GitHub; o identificador testado dos binários permanece 3f4.

DVC não está configurado neste projeto. O grafo mecânico final do corte Synapses foi construído sobre 16 corpora: 147 nós e 1.364 arestas, geração 15. O lint retornou ok como execução, mas reteve 497 endpoints restritos e nove sobreposições de planos históricos/atuais. Não houve alegação de um grafo global sem pendências.

PR #1 · entrega CESEB · Repositório público pessoal. A aceitação técnica relatada cobre as entradas, escalas, sistemas e pacotes declarados; não encerra a sessão por inferência.

RELATÓRIO DE TRABALHO / 19

Uso, limites e lições

Como executar e o que este primeiro produto permite concluir

Operação básica

No Windows, extrair o ZIP inteiro e executar bin/ceseb.exe. No Ubuntu, instalar o DEB e abrir o launcher CESEB. Bibliotecas, plugins e licenças devem acompanhar o executável. Preencher valor, taxa, tipo e prazo; calcular; comparar os cartões; corrigir qualquer erro e recalcular. Editar a entrada limpa resultados antigos; Limpar reinicia o formulário.

Um roteiro reproduzível é R$ 1.000,00, 12% nominal e três meses: Price total R$ 1.020,07 e SAC R$ 1.020,00. Trocar para efetiva invalida o resultado; recalcular deve produzir R$ 1.019,04 e R$ 1.018,98. A ajuda local descreve o modelo e abre os materiais de licença.

Limites do primeiro recorte

O produto não calcula CET, seguros, tarifas, tributos, indexadores, carência ou convenções específicas de bancos. Não persiste simulações, não exporta CSV e não apresenta a tabela mensal completa. O parecer não garante todas as resoluções, distribuições Linux, versões Windows ou acessibilidade por tecnologias assistivas não exercitadas. Não houve scanner remoto, ensaio ABI ou run de CI remoto neste corte.

Lições que o trabalho deixou

Precisão é parte do contrato. Muitas casas decimais não substituem uma regra de fronteira com prova e testes.

UIA não substitui leitura de tela. Valores corretos na árvore acessível podem estar invisíveis ao usuário; presença geométrica e legibilidade exigem evidências diferentes.

Pacote é produto. Launcher, links, runtime, licenças e fonte correspondente precisam ser ensaiados sobre o artefato entregue.

Autoridade e maturidade precisam de aplicação real. Aprovar o corpo, executar o plano, julgar o requisito e confirmar push/merge são operações distintas. A cobrança do usuário revelou falhas de processo e orientou sua correção.

Histórico não é lixo a limpar. Preservar tentativas e cortes permite entender por que o resultado final é confiável, sem fingir que todas as etapas passaram na primeira tentativa.

RELATÓRIO DE TRABALHO / 20

Fontes e trilhas de leitura

Referências clicáveis ao corte documental b68e60a; consulta em 27/10/2026

Tema	Fonte do projeto
Necessidade / descoberta	documents/1003/r1-discovery-proposal.md
Escolhas humanas	documents/1005/r1-human-scope-approval.md
Índice de arquitetura	documents/1015/r3-research-driven-design.md
Refinamento dos requisitos	decisions/1018/r1-requirement-conformance-correction.md
Método numérico	documents/1023/r1-numerical-resolution.md
Escolha do Qt	documents/1024/r1-qt-baseline-review.md
Planos iniciais e cobertura	documents/1033/r1-engineering-plan-review-index.md
WSL / revisão normativa	decisions/1037/r1-wsl-linux-validation-choice.md
Regressões F3/F4	plans/2005/r1-f3-f4-regressions.md
Produto F3/F4	plans/2006/r1-f3-f4-corrections.md
BASE reproduzido	evidence/dispatch/r4/tests/base-result.md
Execução final dos testes	evidence/dispatch/r4/tests/final-result.md
Execução final de produto	evidence/dispatch/r4/implementation/result.md
Revisão independente	evidence/synapses/r4/dev-review.md
Reconciliação	evidence/synapses/r4/analyst.md
Publicação observada	evidence/synapses/r4/syncer.md

Complementos: guia de uso; build e testes; avisos de terceiros; lockfile. O catálogo da seção 5 liga cada requisito à sua fonte exata.

Este relatório é uma síntese editorial do trabalho e não substitui os contratos ou os registros originais. Afirmações de execução são atribuídas aos relatórios do corte indicado. As fontes externas das pesquisas estão conservadas nos documentos 3001–3003 e 1023–1024; não se alega nova pesquisa bibliográfica neste documento. Capturas: originais docs/validation/r4, sem editar o conteúdo do app.